

Deep Learning Practical 1: Model answers

Jupyter, Colab and the first classifier

PyTorch (and optional JAX) preliminaries

What differences do you notice between the PyTorch and JAX examples?

Both libraries express array calculations through shapes, indexing, broadcasting and matrix multiplication. A visible difference is how they change an array: PyTorch allows assignments such as `x[0] = 1`, whereas JAX uses `x = x.at[0].set(1)` to create an updated array. The numerical operations in these introductory examples have the same mathematical meaning despite differences in their names and syntax.

1 Train a first digit classifier

Images, labels and batches

Display a different example. Find its brightest and darkest pixels with `min()` and `max()`. What does `images[:64].shape` return? Which axis selects the image, and which axes select a pixel?

```
i = 1
plt.imshow(images[i, 0], cmap="gray", vmin=-1, vmax=1)
print(images[i].min().item(), images[i].max().item())
print(images[:64].shape)
```

For this example the minimum and maximum are -1 and 1 , and the batch shape is `torch.Size([64, 1, 28, 28])`. Its axes represent image, channel, row and column in that order, so `images[i, 0, row, col]` selects one pixel from image i .

Turning pixels into scores for each class

Explain why the output has shape $(64, 10)$. Count the weights and biases by hand. Before training, should these predictions agree with the labels?

Each of the 64 images receives ten class scores, giving an output of shape $(64, 10)$. Including both biases, the network contains

$$784 \times 16 + 16 + 16 \times 10 + 10 = 12,730$$

parameters. Before training, its random weights bear no useful relationship to the digit labels, so we expect poor predictions. Uniform random guesses would be correct 10% of the time, although an individual untrained network may favour some classes and achieve a different accuracy.

2 Losses and gradient updates

For the first image, inspect the probability assigned to its true label and compare `-probabilities[0, y[0]].log()` with the batch loss. Why are they generally different?

For the first image, the loss is $-\log p_{0,y_0}$. By contrast, `F.cross_entropy(logits, y)` averages this quantity over all 64 images:

$$L = -\frac{1}{64} \sum_{i=0}^{63} \log p_{i,y_i}.$$

Because the model assigns different probabilities to the correct labels of different images, one image's loss will generally differ from that average. For example, a correct-label probability of 0.1 gives a loss of about 2.303, which exceeds the batch loss if the other examples are easier on average.

Match each line of the loop to one operation: choose a batch, predict, measure error, clear old gradients, differentiate, update parameters. Why do the losses sometimes rise between updates?

Code in train	Operation
<code>index = torch.randint(...)</code> and <code>x, y = ...</code>	Choose a batch
<code>logits = model(x)</code>	Predict
<code>loss = F.cross_entropy(logits, y)</code>	Measure error
<code>optimizer.zero_grad()</code>	Clear old gradients
<code>loss.backward()</code>	Differentiate
<code>optimizer.step()</code>	Update parameters

Each update draws a fresh batch, which may be harder than the preceding one, so a rise in batch loss does not necessarily mean that overall performance has worsened. Even on a fixed batch, an update can increase the loss if the step is too large. Clearing gradients before differentiation ensures that the update uses the current batch's derivatives, rather than their sum with derivatives left over from earlier batches.

3 Check the predictions and improve them

Make a new model with 32 hidden units and train it from fresh weights for the same number of updates. Compare the parameter count, validation accuracy and loss. Next change only the learning rate to 0.001 or 0.01. Keep the random seed, batch size and update count fixed. Which change helped, and did it help every digit equally?

Doubling the hidden width gives $784 \times 32 + 32 + 32 \times 10 + 10 = 25,450$ parameters. The comparison below uses 150 updates and batches of 64. We reset the weight seed to 3801 and the batch seed to 3802 for each run so that changing the width leaves the sampled training examples unchanged.

Hidden units	Learning rate	Validation accuracy	Validation cross-entropy
16	0.003	82.86%	0.5562
32	0.003	88.20%	0.4133
32	0.001	85.86%	0.5305
32	0.010	88.06%	0.3982

Widening the model improves both measures in this CPU comparison. Reducing the wider model's learning rate to 0.001 worsens them, while 0.01 gives a slightly lower cross-entropy but almost the same accuracy as 0.003. The benefit is uneven across digits: widening at 0.003 improves accuracy on five from 66.84% to 87.70%, but reduces accuracy on seven from 92.36% to 88.20%. This controlled batch sequence differs from the original run in the practical, which reported 84.52% for its baseline. A wider network can learn more varied features, but its accuracy also depends on how far training progresses in the allotted updates. A small learning rate can leave it undertrained, while a large rate can make the updates unstable. Overall accuracy is an average across digits, so it can improve even when performance on a particular digit worsens.